

TRƯỜNG ĐẠI HỌC KIẾN TRÚC ĐÀ NẴNG (DAU)

Chương 4: Kỹ thuật lập trình đa luồng

Bài giảng · Chương 4

Học thật · làm thật · code thật

TS. Đỗ Phúc Hảo – Giảng viên *Lecturer*

Khoa Công nghệ Thông tin, Trường Đại học Kiến trúc Đà Nẵng

haodp.ai@gmail.com · 0365 674 592 · Đà Nẵng, 2026

Nội dung buổi học *Lecture outline*

1. **Tiến trình và luồng** *process & thread*
2. **Tạo luồng** *Thread & Runnable*
3. **Vòng đời luồng** *lifecycle*
4. **Đồng bộ hóa** *synchronization*
5. **java.util.concurrent**
6. **Tóm tắt và bài tập**

Mục tiêu: làm nhiều việc song song một cách an toàn, giữ cho dữ liệu dùng chung không bị hỏng. Slide tiếng Việt là chính; code Java comment tiếng Anh.



1

Tiến trình và luồng

process & thread

Hai khái niệm nền *Process and thread*

DAU

ĐỊNH NGHĨA · DEFINITION

Tiến trình *process* là một chương trình đang chạy, được hệ điều hành cấp vùng nhớ riêng. **Luồng** *thread* là đơn vị thực thi bên trong tiến trình; nhiều luồng chia sẻ chung vùng nhớ nhưng mỗi luồng có ngăn xếp lời gọi riêng.

Hai tiến trình tách biệt: trao đổi dữ liệu tốn kém.

Các luồng cùng tiến trình: chia sẻ *heap* rất rẻ.

- Mỗi ứng dụng Java đã có sẵn luồng `main` chạy phương thức `main`.
- Máy ảo còn giữ các luồng nền *daemon* như luồng thu gom rác.

Vì sao cần đa luồng *Why multithreading*

DAU

- **Tính đáp ứng** *responsiveness*: tách việc nặng sang luồng phụ, giao diện không bị treo.
- **Tận dụng nhiều nhân**: chỉ chương trình đa luồng mới chạy thật sự song song trên nhiều nhân CPU.
- **Che giấu độ trễ**: một luồng chờ tệp hay mạng, các luồng khác vẫn làm việc tiếp.

LỖI HAY GẶP · PITFALL

Đa luồng không miễn phí: mỗi luồng tốn bộ nhớ ngăn xếp, chuyển luồng *context switch* mất thời gian. Chỉ đa luồng khi có việc thật sự đáng làm song song.

2

Tạo luồng trong Java

Thread & Runnable

Hai con đường mô tả công việc *Two ways*

- **Cách 1**: viết lớp con của `Thread`, ghi đè phương thức `run`.
- **Cách 2**: cài giao tiếp `Runnable` với một phương thức `run`, trao cho một `Thread` để chạy.

GHI NHỚ · KEY POINT

Gọi `start` để sinh luồng mới rồi chạy song song; gọi thẳng `run` chỉ là lời gọi phương thức bình thường trên luồng hiện tại, không có gì song song.

`Runnable` là một giao tiếp hàm *functional interface*, nên có thể viết gọn bằng biểu thức Lambda đã học ở chương 2.

Cách 1: kế thừa lớp Thread *Subclass Thread*

DAU

```java
public class DemNguoc extends Thread {
   private final String ten;
   public DemNguoc(String ten) { this.ten = ten; }

   @Override
   public void run() { // work of this thread
      for (int i = 3; i >= 1; i--) {
         System.out.println(ten + ": " + i);
         try { Thread.sleep(200); } // pause 200 ms
         catch (InterruptedException e) { return; }
      }
   }
}
```

```java
new DemNguoc("A").start();
new DemNguoc("B").start();
// start() launches a new
// thread; the two counters
// run concurrently
```

LỖI HAY GẶP · PITFALL

Đừng gọi `run` trực tiếp.

Cách 2: cài giao tiếp Runnable *Implement Runnable*

DAU

```java
public class ViDuRunnable {
  public static void main(String[] args) {
    Runnable congViec = () -> { // functional interface
      for (int i = 0; i < 3; i++)
        System.out.println("Buoc " + i + " tren "
          + Thread.currentThread().getName());
    };
    Thread t = new Thread(congViec, "Worker");
    t.start(); // launch the worker
    System.out.println("main van chay tiep");
  }
}
```

GHI NHỚ · KEY POINT

Ưu tiên `Runnable`: lớp vẫn kế thừa được lớp khác, và tách việc khỏi cách chạy việc.

Cùng một `Runnable` có thể giao cho `Thread` hay cho `ExecutorService`.



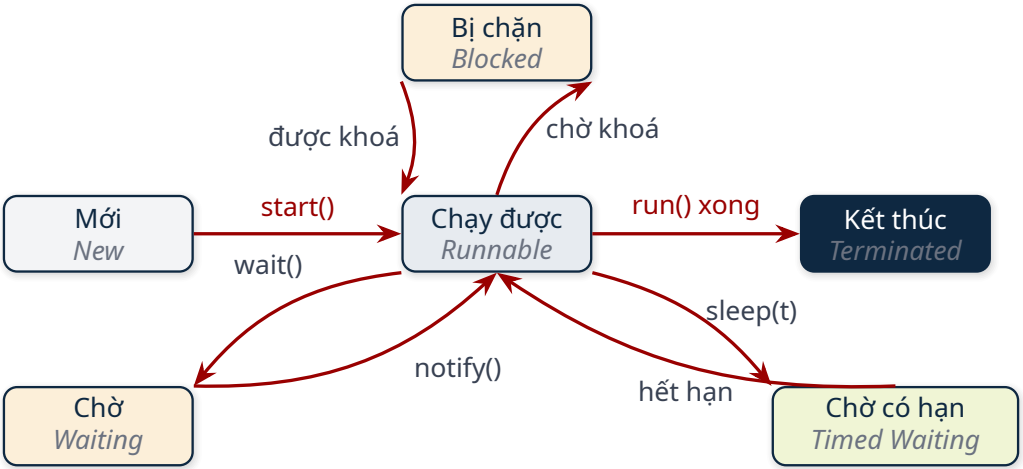
3

Vòng đời của luồng

thread lifecycle

Sơ đồ vòng đời luồng *Thread lifecycle*

DAU



Đọc sơ đồ vòng đời *Reading the diagram*

DAU

- **Mới** *New*: vừa tạo bằng `new`, chưa chạy.
- **Chạy được** *Runnable*: sau `start`, sẵn sàng và chờ hệ điều hành cấp CPU.
- **Bị chặn** *Blocked*: cần một khoá đang bị luồng khác giữ.
- **Chờ** *Waiting*: gọi `wait`, đợi tới khi luồng khác gọi `notify`.
- **Chờ có hạn** *Timed Waiting*: gọi `sleep` hay `join` có thời hạn, tự quay lại khi hết giờ.
- **Kết thúc** *Terminated*: `run` xong, không khởi chạy lại được.

Java mô hình hóa các trạng thái này trong kiểu liệt kê `Thread.State`.

Thread vs Runnable vs ExecutorService *Comparison*

DAU

Tiêu chí	Kế thừa `Thread`	Cài `Runnable`	`ExecutorService`
Cách dùng	Lớp con, ghi đè `run`	Viết `run`, trao cho `Thread`	Nộp tác vụ cho nhóm
Suất kế thừa	Tốn mất	Còn nguyên	Còn nguyên
Tái dùng luồng	Không	Không tự có	Có, bể luồng *pool*
Trả về kết quả	Không trực tiếp	Không trực tiếp	Có, qua `Future`
Khi nào dùng	Học, ví dụ nhỏ	Tách việc khỏi luồng	Ứng dụng thật

GHI NHỚ · KEY POINT

Càng lên bậc trừu tượng cao, mã càng gọn và an toàn hơn.

4

Đồng bộ hóa
synchronization

Tranh chấp dữ liệu *Race condition*

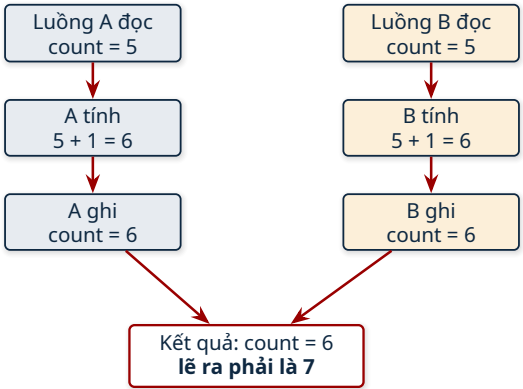
ĐỊNH NGHĨA · DEFINITION

Tranh chấp dữ liệu *race condition*: tính đúng đắn của chương trình phụ thuộc vào thứ tự xen kẽ ngẫu nhiên giữa các luồng cùng truy cập một dữ liệu chia sẻ, mà ít nhất một luồng thực hiện thao tác ghi.

- Lệnh `count++` không nguyên tử: nó là đọc, cộng một, rồi ghi lại.
- Hai luồng cùng đọc giá trị cũ trước khi luồng nào kịp ghi: một lần tăng bị mất.

Hai luồng tranh chấp một biến đếm *Lost update*

hai luồng xen kẽ trên biến chung count



Bộ đếm không an toàn *Unsafe counter*

DAU

```
1  public class DemKhongAnToan {
2    private int count = 0;
3    public void tang() { count++; } // not atomic
4    public int getCount() { return count; }
5
6    public static void main(String[] a)
7        throws InterruptedException {
8      DemKhongAnToan d = new DemKhongAnToan();
9      Runnable job = () -> {
10       for (int i = 0; i < 100_000; i++) d.tang();
11     };
12     Thread t1 = new Thread(job), t2 = new Thread(job);
13     t1.start(); t2.start();
14     t1.join(); t2.join(); // wait both to finish
15     System.out.println(d.getCount());
16   }
17 }
```

LỖI HAY GẶP · PITFALL

Kết quả thường nhỏ hơn 200000 và đổi mỗi lần chạy: dấu hiệu kinh điển của tranh chấp.

Từ khoá synchronized *The synchronized keyword*

DAU

```java
public class DemAnToan {
  private int count = 0;
  // only one thread at a time
  public synchronized void tang() {
    count++;
  }
  public synchronized int getCount() {
    return count;
  }
}
// result is always 200000
```

ĐỊNH NGHĨA · DEFINITION

Vùng tới hạn *critical section*: đoạn mã mà tại mỗi thời điểm chỉ một luồng được vào.

`synchronized` dựa trên khoá nội tại *intrinsic lock* của mỗi đối tượng. Có thể đồng bộ cả phương thức, hoặc một khối `synchronized (khoa) {...}` hẹp hơn nên nhanh hơn.

Khoá tường minh ReentrantLock *Explicit lock*

DAU

```java
import java.util.concurrent.locks.ReentrantLock;

public class DemBangKhoa {
  private final ReentrantLock lock
      = new ReentrantLock();
  private int count = 0;
  public void tang() {
    lock.lock(); // acquire
    try { count++; } // critical section
    finally { lock.unlock(); } // always release
  }
}
```

GHI NHỚ · KEY POINT

`ReentrantLock` linh hoạt hơn `synchronized`: thử giành khoá có thời hạn, giành khoá ngắt được.

LỖI HAY GẶP · PITFALL

Phải tự nhả khoá. Luôn đặt `unlock` trong khối `finally` để khoá được nhả dù có ngoại lệ.

Từ khoá volatile *Visibility*

DAU

- Mỗi luồng có thể giữ bản sao biến trong vùng đệm riêng, không thấy thay đổi của luồng khác.
- Đánh dấu trường là `volatile` buộc mọi lần đọc và ghi đi thẳng vào bộ nhớ chính.

LỖI HAY GẶP · PITFALL

`volatile` chỉ bảo đảm tính nhìn thấy *visibility*, không bảo đảm tính nguyên tử *atomicity*. Nó hợp với cờ dừng kiểu `boolean`, không thay được `synchronized` cho `count++`.

Dùng `volatile` cho cờ trạng thái; dùng `synchronized` hoặc khoá cho thao tác gồm nhiều bước.

Khoá chết *Deadlock*

ĐỊNH NGHĨA · DEFINITION

Khoá chết *deadlock* xảy ra khi hai luồng mỗi luồng giữ một khoá rồi cùng chờ khoá mà luồng kia đang giữ; không luồng nào nhả, cả hai đứng im mãi mãi.

A giữ khoá 1, chờ khoá 2.
B giữ khoá 2, chờ khoá 1.

Phòng tránh: mọi luồng giành nhiều khoá theo cùng một thứ tự cố định.

GHI NHỚ · KEY POINT

Nếu ai cũng lấy khoá 1 trước rồi mới tới khoá 2, vòng chờ vòng tròn không thể hình thành.



5

Gói java.util.concurrent

high-level concurrency

ExecutorService và bể luồng *Thread pool*

DAU

```java
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ViDuExecutor {
  public static void main(String[] args) {
    ExecutorService pool =
      Executors.newFixedThreadPool(3); // 3 workers
    for (int i = 1; i <= 6; i++) {
      int id = i;
      pool.submit(() -> System.out.println(
        "Tac vu " + id + " tren "
        + Thread.currentThread().getName()));
    }
    pool.shutdown(); // finish queued tasks then stop
  }
}
```

ĐỊNH NGHĨA · DEFINITION

Bể luồng *thread pool* tái dùng một số luồng cố định để chạy lần lượt các tác vụ trong hàng đợi.

Tránh chi phí tạo, huỷ luồng liên tục và khống chế số luồng đồng thời.

Callable và Future *Returning a result*

DAU

```java
import java.util.concurrent.*;
public class ViDuFuture {
  public static void main(String[] a)
      throws Exception {
    ExecutorService pool =
      Executors.newFixedThreadPool(2);
    Callable<Integer> task = () -> { // returns value
      int s = 0;
      for (int i = 1; i <= 100; i++) s += i;
      return s; // 1 + 2 + ... + 100
    };
    Future<Integer> f = pool.submit(task);
    System.out.println("Tong = " + f.get());
    pool.shutdown(); // get() waits for result
  }
}
```

GHI NHỚ · KEY POINT

`Runnable` không trả kết quả. `Callable<V>` có `call` trả kiểu `V` và được ném ngoại lệ.

`Future<V>` là phiếu hẹn lấy kết quả; `get` chờ tới khi tác vụ xong rồi trả về giá trị.

Hàng đợi chặn *Blocking queue*

DAU

```java
import java.util.concurrent.*;
public class SanXuatTieuThu {
  public static void main(String[] args) {
    BlockingQueue<Integer> q =
      new ArrayBlockingQueue<>(5); // capacity 5
    Runnable producer = () -> {
      try { for (int i = 1; i <= 5; i++) q.put(i); }
      catch (InterruptedException e) {} // blocks if full
    };
    Runnable consumer = () -> {
      try { for (int i = 1; i <= 5; i++)
        System.out.println("Tieu thu " + q.take()); }
      catch (InterruptedException e) {} // blocks if empty
    };
    new Thread(producer).start();
    new Thread(consumer).start();
  }
}
```

ĐỊNH NGHĨA · DEFINITION

Hàng đợi chặn *blocking queue*: rỗng thì luồng lấy ra tự chờ; đầy thì luồng bỏ vào tự chờ.

Nền tảng tự nhiên cho mẫu sản xuất tiêu thụ *producer consumer*.

6

Tóm tắt và bài tập

wrap-up & exercises

Tóm tắt chương *Summary*

DAU

- Luồng chia sẻ chung vùng nhớ trong một tiến trình; chia sẻ rẻ nhưng dễ sai.
- Tạo luồng bằng kế thừa `Thread` hoặc cài `Runnable`; gọi `start`, không gọi `run`.
- Vòng đời: Mới, Chạy được, Bị chặn, Chờ, Chờ có hạn, Kết thúc.
- Tranh chấp dữ liệu sửa bằng `synchronized`, `ReentrantLock`; `volatile` chỉ cho tính nhìn thấy.
- Khoá chết: phòng tránh bằng thứ tự giành khoá cố định.
- `java.util.concurrent`: `ExecutorService`, `Callable`/`Future`, hàng đợi chặn.

GHI NHỚ · KEY POINT

Quy ước hiện đại: ưu tiên `java.util.concurrent`; tự dựng luồng và khoá chỉ để học cơ chế.

Bài tập về nhà *Homework*

1. Giải thích vì sao gọi `start` chứ không phải `run` để chạy luồng song song; điều gì xảy ra nếu gọi nhầm `run`.
2. Cho biết `volatile` bảo đảm và không bảo đảm điều gì; vì sao không thay được `synchronized` cho `count++`.
3. **(Lập trình)** Viết lớp `TaiKhoan` có `napTien` và `rutTien`. Cho mười luồng cùng thao tác; chạy bản không đồng bộ thấy số dư sai, rồi sửa bằng `synchronized`.
4. **(Lập trình)** Dùng `ExecutorService` bể bốn luồng tính song song tổng bình phương các số từ 1 đến 1000, chia bốn khoảng, thu kết quả qua `Future`.
5. **(Lập trình)** Cài mẫu sản xuất tiêu thụ bằng `BlockingQueue`: hai luồng sản xuất, ba luồng tiêu thụ, dùng giá trị quy ước báo hết để dừng.

CẢM ƠN

Hỏi đáp · Thảo luận

TS. Đỗ Phúc Hảo

haodp.ai@gmail.com · 0365 674 592 · Khoa CNTT, ĐH Kiến trúc Đà Nẵng